

Proposta Técnica

Plataforma Centralizada de Comunicação e Notificações Corporativas

1. Visão Geral

Este documento apresenta a proposta de desenvolvimento de uma plataforma centralizada de comunicação responsável pelo gerenciamento e envio de notificações utilizadas pelos sistemas corporativos.

A solução tem como objetivo criar uma camada única de comunicação entre os sistemas internos e os diferentes canais de entrega, permitindo o envio de mensagens sem que cada sistema precise implementar regras próprias de integração com provedores de email ou outros serviços.

A plataforma funcionará como um serviço independente (Notification Service), disponibilizando uma API para que aplicações internas possam solicitar envios de mensagens.

2. Objetivos do Projeto

Objetivo Principal

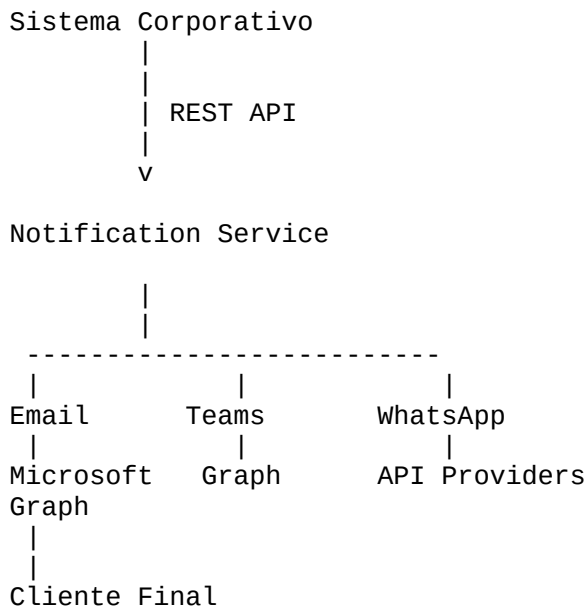
Desenvolver um serviço corporativo capaz de realizar o envio de notificações através de múltiplos canais, inicialmente com suporte a email utilizando Microsoft Graph API.

Objetivos Específicos

- Centralizar o gerenciamento de emails corporativos.
 - Remover dependência direta dos sistemas com Outlook, SMTP ou credenciais individuais.
 - Padronizar templates de comunicação.
 - Criar histórico e auditoria dos envios.
 - Permitir integração com múltiplos sistemas existentes.
 - Facilitar futuras integrações com novos canais de comunicação.
 - Melhorar segurança através da centralização de autenticação.
-

3. Arquitetura da Solução

Visão Macro



4. Funcionamento Geral

Fluxo de envio:

1. Sistema corporativo solicita uma notificação.
2. Notification Service recebe a requisição.
3. Serviço valida dados.
4. Identifica o canal de comunicação.
5. Processa template da mensagem.
6. Realiza autenticação com o provedor.
7. Executa envio.
8. Registra histórico.
9. Retorna status para o sistema solicitante.

5. Tecnologias Propostas

Backend

- Python

- FastAPI
- Pydantic
- SQLAlchemy ou MongoDB
- Redis para processamento assíncrono
- Docker

Integrações

- Microsoft Graph API
- OAuth 2.0
- REST APIs

Banco de Dados

Armazenamento:

- Logs de envio
 - Templates
 - Configurações
 - Histórico
 - Auditoria
-

6. Estrutura do Projeto

notification-service

app/

```
├── api/
│   ├── routes/
│   │   ├── email_routes.py
│   │   ├── notification_routes.py
│   │   └── health_routes.py
├── core/
│   ├── config.py
│   ├── security.py
│   └── exceptions.py
├── services/
│   ├── email_service.py
│   ├── template_service.py
│   ├── notification_service.py
│   └── audit_service.py
├── providers/
│   └── microsoft_graph.py
```

```
├── smtp_provider.py
├── whatsapp_provider.py
├── models/
│   ├── notification.py
│   ├── template.py
│   └── user.py
├── repositories/
│   ├── notification_repository.py
│   └── template_repository.py
├── workers/
│   └── email_worker.py
└── main.py
```

7. Principais Classes do Sistema

NotificationController

Responsável por receber solicitações externas.

Responsabilidades:

- Receber requisições HTTP.
- Validar dados.
- Encaminhar processamento.

Métodos:

`create_notification()`

`get_notification_status()`

EmailService

Responsável pelas regras de envio.

Responsabilidades:

- Preparar mensagens.
- Definir provedor.
- Executar envio.

Métodos:

`send_email()`

`validate_email()`

`process_template()`

MicrosoftGraphProvider

Responsável pela comunicação com Microsoft Graph API.

Responsabilidades:

- Gerenciar autenticação OAuth2.
- Obter tokens.
- Executar chamadas para API Microsoft.

Métodos:

`authenticate()`

`send_message()`

`refresh_token()`

TemplateService

Responsável pela criação das mensagens.

Responsabilidades:

- Carregar templates.
- Substituir variáveis.
- Gerar HTML final.

Métodos:

`load_template()`

`render_template()`

NotificationRepository

Responsável pela persistência.

Métodos:

`save()`

`find_by_id()`

update_status()

AuditService

Responsável pelo rastreamento.

Registra:

- Sistema origem.
 - Usuário responsável.
 - Data/hora.
 - Destinatário.
 - Resultado.
-

8. Modelo de Dados

Notification

id

origin_system

channel

recipient

subject

template

status

created_at

sent_at

error_message

Template

id

name

description

html_content

variables
created_at

9. API Disponibilizada

Solicitar envio

POST

/api/v1/notifications

Exemplo:

```
{
  "channel": "email",
  "recipient": "cliente@email.com",
  "template": "resultado_disponivel",
  "data": {
    "cliente": "João",
    "numero": "12345"
  }
}
```

Resposta:

```
{
  "id": "987654",
  "status": "PROCESSING"
}
```

10. Processamento Assíncrono

Para grandes volumes será utilizado processamento em fila.

Arquitetura:

Sistema

|

API

|

Queue

|

Worker

|

Provider Email

|

Cliente

Benefícios:

- Maior desempenho.
 - Evita bloqueio do sistema.
 - Permite reenvio automático.
 - Controle de falhas.
-

11. Segurança

Implementações:

- OAuth 2.0 Microsoft.
 - Controle de acesso por sistema.
 - API Keys internas.
 - Logs de auditoria.
 - Criptografia de credenciais.
 - Controle de permissões.
-

12. Funcionalidades Futuras

Microsoft Teams

Envio de:

- Alertas.
 - Avisos.
 - Mensagens em canais.
-

WhatsApp Business

Possibilidade de:

- Comunicação com clientes.
- Atualizações de status.

- Alertas automáticos.
-

Dashboard Administrativo

Recursos:

- Histórico de mensagens.
 - Taxa de sucesso.
 - Monitoramento de falhas.
 - Gestão de templates.
-

Inteligência Artificial

Possibilidades:

- Geração automática de mensagens.
 - Classificação de prioridade.
 - Sugestão de respostas.
 - Detecção de falhas.
-

13. Benefícios para a Organização

Redução de Complexidade

Sistemas deixam de implementar integrações individuais.

Segurança

Credenciais ficam centralizadas.

Padronização

Todos os sistemas utilizam o mesmo padrão de comunicação.

Escalabilidade

Novos sistemas podem utilizar o serviço sem novas integrações.

Manutenção Simplificada

Alterações de provedores ou regras acontecem em apenas um ponto.

14. Roadmap de Desenvolvimento

Fase 1 - MVP

- API de envio de email.
- Integração Microsoft Graph.
- Templates.
- Logs.
- Docker.

Fase 2

- Filas de processamento.
- Dashboard.
- Controle de usuários.

Fase 3

- Teams.
 - WhatsApp.
 - SMS.
 - Inteligência Artificial.
-

Conclusão

A Plataforma Centralizada de Comunicação permitirá que os sistemas corporativos utilizem uma única solução para envio de notificações, reduzindo acoplamento, aumentando segurança e criando uma base preparada para futuras integrações.

A solução transforma o envio de mensagens em um serviço corporativo reutilizável, permitindo que novos sistemas sejam integrados rapidamente sem necessidade de desenvolver novamente toda a infraestrutura de comunicação.